Keras 3.x Technical Setup Guide

Overview

This guide provides step-by-step instructions for installing and configuring **Keras 3.x with the PyTorch backend** for Week 6 of the ML course. Follow these instructions carefully to avoid common setup pitfalls. A companion **Troubleshooting Guide** covers what to do when something goes wrong.

Target configuration

- Keras 3.x (NOT Keras 2.x)
- PyTorch backend (NOT TensorFlow backend)
- Python 3.9+ (3.10 recommended)
- Runs on CPU; NVIDIA or Intel GPU is optional

Estimated setup time: 15–30 minutes

> **The single most important rule:** every command below assumes your virtual environment (`ml_week6_env`) is **activated**. If your shell prompt does not start with `(ml_week6_env)`, re-activate it (Step 2) before running any `pip` or `python` command.

Quick Start (For Experienced Users)

If you are comfortable with Python environments, the whole setup is four steps. Run them **inside an activated virtual environment**.

IMPORTANT PYTHON AND BASH Commands should be run from your terminal within the course Directory

```
# 1. Create and activate a virtual environment
python -m venv ml_week6_env
source ml_week6_env/bin/activate   # Windows: ml_week6_env\Scripts\activate

# 2. Install PyTorch - run ONE line for your hardware (details in Step 3)
# CPU:
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
# NVIDIA GPU (CUDA 12.8):
# pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128
# Intel GPU (XPU):
# pip install torch torchvision --index-url https://download.pytorch.org/whl/xpu

# 3. Install Keras 3.x and supporting libraries
pip install "keras>=3.0" scikit-learn numpy matplotlib jupyter

# 4. Set the backend, then verify
export KERAS_BACKEND=torch   # Windows PowerShell: $env:KERAS_BACKEND="torch"
python -c "import keras; print(keras.__version__, keras.backend.backend())"
```

Expected output: `3.x.x torch` — the Keras version followed by the active backend.

If this works, you are done. If not, follow the detailed instructions below and consult the Troubleshooting Guide.

Detailed Setup Instructions

Step 1 — Check your Python version

Requirement: Python 3.9 or higher (3.10 recommended).

```
python --version
```

Expected output: `Python 3.10.x` (any 3.9–3.12 is fine).

If Python is too old or missing, download it from https://www.python.org/downloads/ (Windows/macOS) or install it through your system package manager (Linux), then re-check.

> On some systems the command is `python3` instead of `python`. If `python --version` fails, try `python3 --version` and use `python3` everywhere below.

Step 2 — Create and activate a virtual environment

Why? A virtual environment isolates this course's packages from the rest of your system, preventing version conflicts. **We use one environment, `ml_week6_env`, for the entire course.**

```
# Create the environment (run once)
python -m venv ml_week6_env
```

Activate it (you must do this **every time** you open a new terminal):

```
# macOS / Linux
source ml_week6_env/bin/activate

# Windows (PowerShell)
ml_week6_env\Scripts\Activate.ps1

# Windows (Command Prompt)
ml_week6_env\Scripts\activate.bat
```

Verify activation — your prompt should now begin with `(ml_week6_env)`:

```
# macOS / Linux
which python      # path should contain "ml_week6_env"

# Windows
where python      # path should contain "ml_week6_env"
```

Before installing anything, upgrade pip inside the environment:

```
python -m pip install --upgrade pip
```

> **To leave the environment later:** run `deactivate`. To return, just re-run the activate command above — you do **not** create it again.

Step 3 — Install PyTorch (choose your hardware)

> **CRITICAL:** PyTorch must be installed **before** Keras, and you must pick the wheel that matches your hardware. A plain `pip install torch` does **not** reliably give you GPU support — PyTorch publishes a separate build for each accelerator, selected with `--index-url`. Run **only one** of the options below.

A virtual environment can hold only one PyTorch build at a time. If you later switch hardware, the simplest path is a fresh environment rather than reinstalling over the top.

Option A — CPU only (recommended; sufficient for all Week 6 work) Works on any machine, including Macs (Apple Silicon GPU acceleration is included automatically in the CPU wheel via the MPS backend).

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Time: 2–5 minutes depending on connection speed.

Option B — NVIDIA GPU (CUDA) Only if you have an NVIDIA GPU **and** an up-to-date NVIDIA driver. You do **not** need to install the CUDA Toolkit separately — the wheel bundles the CUDA runtime. You **do** need a driver new enough for your chosen CUDA version.

First confirm the GPU and driver are visible:

```
nvidia-smi
```

Then install the wheel whose CUDA version your driver supports. When unsure, pick the **CPU option** or the lowest CUDA version, or use the interactive selector at https://pytorch.org/get-started/locally/.

```
# CUDA 12.6
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126

# CUDA 12.8 (good default for recent drivers)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128

# CUDA 13.0 (newest drivers only)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu130
```

> The index version (`cu126`, `cu128`, `cu130`, …) selects the bundled CUDA runtime. Your installed NVIDIA driver must be at least as new as that runtime requires; the driver version shown by `nvidia-smi` does **not** need to match exactly.

Option C — Intel GPU (XPU) For Intel Arc and other recent Intel GPUs, use the native XPU wheel:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/xpu
```

Prerequisites: install the Intel GPU driver **first** (see https://pytorch.org/docs/stable/notes/get_start_xpu.html). The required Intel runtime libraries are pulled in automatically with the wheel.

> **Note:** the older separate `intel-extension-for-pytorch` package is being retired (end of life ~March 2026). Intel GPU support is now built into native PyTorch, so the XPU index above is the correct, current path — do **not** install `intel-extension-for-pytorch` for this course.

Verify PyTorch and detect your accelerator Run this in a Python session (type `python`, paste the block, press Enter) or in a Jupyter cell:

```
import torch
print(f"PyTorch {torch.__version__}")
print("CUDA (NVIDIA):", torch.cuda.is_available())
print("XPU (Intel): ", hasattr(torch, "xpu") and torch.xpu.is_available())
mps = getattr(torch.backends, "mps", None)
print("MPS (Apple): ", bool(mps) and mps.is_available())
```

Expected output (CPU-only machine):

```
PyTorch 2.x.x
CUDA (NVIDIA): False
XPU (Intel):  False
MPS (Apple):  False
```

On a GPU machine, the matching line should read `True`. If you installed a GPU wheel but see `False`, your driver is the usual culprit — see the Troubleshooting Guide. CPU training is perfectly fine for Week 6, so a `False` here is not a blocker.

Step 4 — Install Keras 3.x

```
pip install "keras>=3.0"
```

> **IMPORTANT:** This installs **standalone Keras 3.x**, not the `tensorflow.keras` (Keras 2.x) that ships inside TensorFlow. If TensorFlow is present in this environment it can shadow Keras 3 — see the Troubleshooting Guide ("Keras 2.x installed instead of 3.x").

Time: 1–2 minutes.

Verify:

```
python -c "import keras; print(f'Keras {keras.__version__}')"
```

Expected output: `Keras 3.x.x`.

Step 5 — Install supporting libraries

```
pip install scikit-learn numpy matplotlib jupyter
```

What these are for:

- **scikit-learn** — metrics, preprocessing, and classic ML baselines
- **numpy** — array operations
- **matplotlib** — plotting training history
- **jupyter** — interactive notebooks

Step 6 — Configure the Keras backend

CRITICAL: the Keras backend must be selected **before** `keras` is imported. There are two ways to do this; pick one.

Per-script (recommended for the course): set the environment variable at the very top of every notebook/script, before importing keras.

```
import os
os.environ['KERAS_BACKEND'] = 'torch'   # MUST come before importing keras

import keras                             # now safe to import
```

Order matters:

```
#  CORRECT
import os
os.environ['KERAS_BACKEND'] = 'torch'
import keras

#  WRONG - backend is already locked in by the time you set the variable
import keras
os.environ['KERAS_BACKEND'] = 'torch'   # too late!
```

Permanent (optional): set it once in your shell so you never forget. With the variable set this way you can skip the `os.environ` line in your code.

```
# macOS / Linux - add to ~/.bashrc or ~/.zshrc
export KERAS_BACKEND=torch

# Windows (PowerShell, current session)
$env:KERAS_BACKEND = "torch"
```

Step 7 — Complete verification

Run this comprehensive test in a Jupyter notebook (launch with `jupyter notebook` from your activated environment):

```python
import os
os.environ['KERAS_BACKEND'] = 'torch'   # must be first!

import keras
import torch
import numpy as np
import matplotlib.pyplot as plt

print("=" * 50)
print("KERAS SETUP VERIFICATION")
print("=" * 50)

# Versions
print(f"Python:  {__import__('sys').version.split()[0]}")
print(f"Keras:   {keras.__version__}")
print(f"PyTorch: {torch.__version__}")
print(f"NumPy:   {np.__version__}")
print(f"Backend: {keras.backend.backend()}")
print(f"CUDA (NVIDIA): {torch.cuda.is_available()}")
print(f"XPU (Intel):   {hasattr(torch,'xpu') and torch.xpu.is_available()}")

# MNIST loading
print("\nTesting MNIST data loading...")
from keras.datasets import mnist
(X_train, y_train), (X_test, y_test) = mnist.load_data()
print(f"MNIST loaded: {X_train.shape[0]} training, {X_test.shape[0]} test samples")

# Model creation
print("\nTesting model creation...")
model = keras.Sequential([
    keras.layers.Input(shape=(28, 28)),
    keras.layers.Flatten(),
    keras.layers.Dense(10, activation='softmax'),
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
print("Model created and compiled")

# Quick training test
print("\nTesting training (1 epoch on 1000 samples)...")
X_small = X_train[:1000] / 255.0
y_small = y_train[:1000]
history = model.fit(X_small, y_small, epochs=1, verbose=0)
print(f"Training works - loss {history.history['loss'][0]:.4f}, "
      f"accuracy {history.history['accuracy'][0]:.4f}")

print("\n" + "=" * 50)
print("SUCCESS! Your Keras setup is working correctly.")
print("=" * 50)
```

Expected output:

```
==================================================
KERAS SETUP VERIFICATION
==================================================
Python:  3.10.x
Keras:   3.x.x
PyTorch: 2.x.x
```

```
NumPy:   2.x.x
Backend: torch
CUDA (NVIDIA): False
XPU (Intel):   False

Testing MNIST data loading...
MNIST loaded: 60000 training, 10000 test samples

Testing model creation...
Model created and compiled

Testing training (1 epoch on 1000 samples)...
Training works - loss 0.xxxx, accuracy 0.xxxx

================================================
SUCCESS! Your Keras setup is working correctly.
================================================
```

If all checks pass, you are ready for Week 6. If any check fails, open the **Troubleshooting Guide** and find the matching issue.

Daily workflow reminder

Each time you sit down to work:

1. Open a terminal and **activate the environment**: `source ml_week6_env/bin/activate` (Windows: `ml_week6_env\Scripts\activate`). Confirm the `(ml_week6_env)` prefix.
2. Launch Jupyter: `jupyter notebook`.
3. At the top of every notebook, set the backend **before** importing keras:

```
import os
os.environ['KERAS_BACKEND'] = 'torch'
import keras
```

4. When finished, `deactivate`.

Quick reference — install commands by hardware

Run the PyTorch line for your hardware **inside the activated `ml_week6_env`**.

CPU (any machine, including Apple Silicon):

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

NVIDIA GPU — use the line matching your CUDA version:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu130
```

Intel GPU (Arc / XPU):

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/xpu
```

After PyTorch, the rest is identical on every machine:

```
pip install "keras>=3.0" scikit-learn numpy matplotlib jupyter
```